

PlantFolio — Project Proposal

Where Your Plants Tell Their Story

1. Executive Summary

PlantFolio is a mobile-responsive web platform where plant lovers build a profile for every plant they own, track its health, trade plants with others, and diagnose problems from a photo. It fulfils the coursework brief for a **Plant Care & Exchange Community** — plant profiles and care information, exchange listings, user collections and care reminders — and extends it with two machine learning features: automatic **plant species recognition** and **plant disease detection**.

The platform combines three ideas that usually live in separate places: the visual, social feel of Instagram, the collection-keeping of a personal catalogue, and the practical help of a care assistant. Every plant gets its own story — photos, health history, care schedule — which is where the tagline *"Where Your Plants Tell Their Story"* comes from.

The project will be delivered in four phases over roughly 12 weeks. The core platform (PHP, MySQL, HTML/CSS/JavaScript) is built first and is complete on its own; the ML features (Python) are layered on afterwards as a separate service, so they add value without putting the required features at risk.

Approval is requested to proceed with this scope, including a Python-based ML service running alongside the PHP application.

2. Problem Statement and Opportunity

Plant owners today manage their plants across scattered tools, and none of them connects care, community and diagnosis in one place.

- **Care knowledge is fragmented.** Watering notes live in phone reminders, care tips in search results, and photos in a camera roll. There is no single record of how a plant has been doing over time.
- **Plant swapping lacks structure and trust.** Exchanges mostly happen in general social media groups, where listings get buried in feeds and there is no way to see whether a trader is reliable.

- **Identifying plants and problems needs expertise.** Beginners often do not know what species they own, and by the time yellow leaves or spots are recognised as disease, the plant may already be lost.

The opportunity: a dedicated community platform that gives each plant a living profile, turns exchanges into searchable listings backed by user ratings, and uses image-based machine learning to answer the two questions every owner asks — *"What is this plant?"* and *"What is wrong with it?"*

Existing services cover parts of this. [PlantSwap](#) focuses on exchanging plants, and [Plantix](#) focuses on photo-based crop diagnosis. PlantFolio draws on both but brings collection management, community and diagnosis together for individual plant lovers.

3. Project Objectives

The project has six objectives, each tied to something that can be demonstrated at the final presentation.

1. **Deliver every feature in the coursework brief** — plant profiles with care information, exchange listings, user collections and care reminders — as a working, mobile-responsive website.
2. **Build a secure foundation** — hashed passwords, protected sessions, prepared SQL statements and validated file uploads — so user data and images are handled safely.
3. **Create a genuine community** — public plant profiles, username search, and a rating system that makes exchanges trustworthy.
4. **Add image-based plant recognition** so users can identify an unknown plant from a single photo and pre-fill its profile.
5. **Add image-based disease detection** using a Convolutional Neural Network (CNN), returning a diagnosis with a confidence score and care guidance.
6. **Document and present the work professionally** — source code on GitHub, a technical report, and a live demonstration.

4. Proposed Solution and Key Features

PlantFolio is organised around a simple hierarchy: **User → Collection → Plant**. Each user can keep several collections (for example "Balcony" or "Succulents"), and each plant inside them has its own profile and history.

4.1 Core features (coursework scope)

Feature	What the user can do	Brief requirement met
Secure registration and login	Create an account, log in and out, edit a profile with a bio, location and photo	Foundation for all features
Plant profiles	Add, edit and delete plants with photos, species, acquisition date and notes; view species care information (light, water, humidity)	Plant profiles and care information
Collections	Group plants into named collections and share them publicly or keep them private	User collections
Health tracking	Record each plant's health status (Thriving, Stable, Needs attention, Sick) with dated notes and photos, building a timeline	Plant profiles
Care reminders	Set a watering or fertilising interval per plant; a dashboard lists what is due today or overdue	Simple care reminders
Search and discovery	Search and filter plants by species, health status or location, and find users by username	Community
Exchange marketplace	Browse listings → express interest → view the owner's contact details once interest is sent	Plant exchange listings
User ratings	Rate a trader from 1 to 5 after an exchange; average ratings appear on profiles and listings	Community trust
Responsive design	Use every feature comfortably on a phone, tablet or desktop	Usability

4.2 Machine learning enhancements

- **Plant Recognition** — the user uploads a photo and the system suggests the most likely species with a confidence score. Accepting a suggestion fills in the plant's species and care information automatically.
- **Disease Detection** — the user uploads a photo of an affected leaf and a CNN predicts whether it is healthy or which disease it shows, with a confidence score and recommended next steps. Each diagnosis is saved to the plant's health timeline.

Both features are designed as helpers, not authorities: low-confidence results are clearly labelled, and users always confirm before anything is saved.

5. Target Users and User Journeys

PlantFolio serves individual plant lovers; every account has the same features, with no separate business or shop accounts.

User type	Need	How PlantFolio helps
Beginner plant owner	Keep new plants alive and learn what they need	Species care information, reminders, recognition and diagnosis
Collector	Show off and organise a growing collection	Collections, rich plant profiles, health timelines
Swapper	Find new plants and trade surplus cuttings safely	Exchange listings, interest requests, trader ratings

Journey A — adding an unknown plant

1. Log in and choose *Add plant*.
2. Upload a photo; the system suggests a species with its confidence score.
3. Confirm the suggestion; care information and a default watering reminder are filled in.
4. Save the plant into a collection.

Journey B — a plant looks unwell

1. Open the plant's profile and choose *Check health*.
2. Upload a leaf photo and receive a predicted condition with guidance.
3. Save the result; the plant's status and health timeline update.

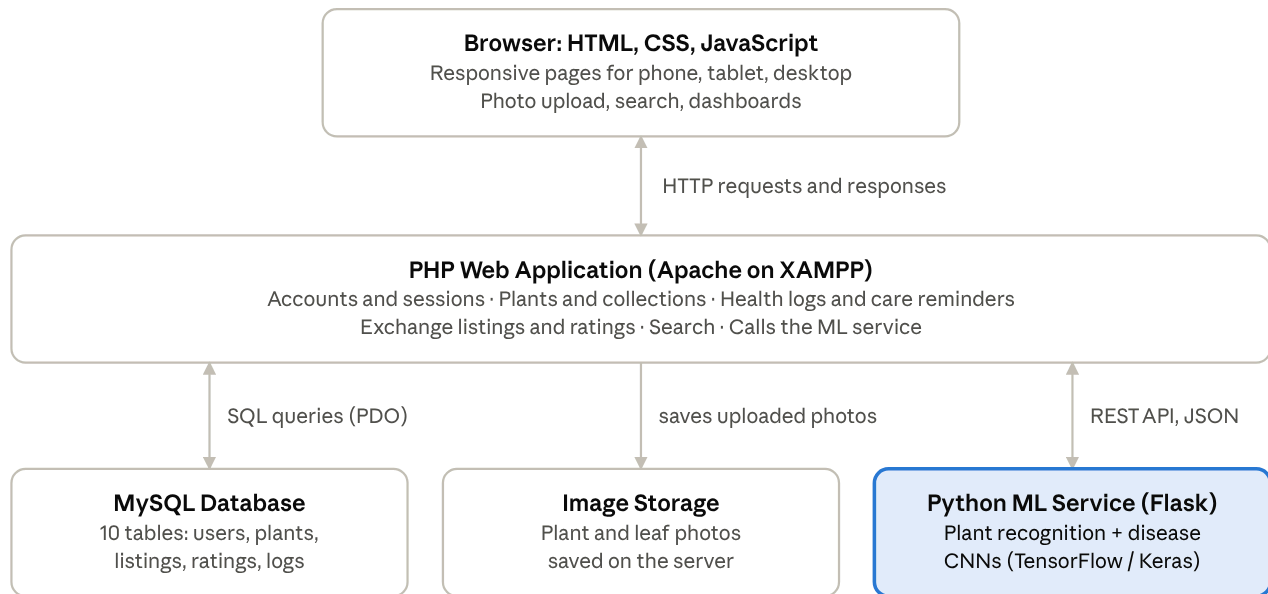
Journey C — trading a plant

1. Browse or filter exchange listings.
2. Press *I'm interested* on a listing; the owner's contact details are revealed.
3. Arrange the swap outside the platform, then rate the other trader.

6. System Architecture and Technology Stack

PlantFolio uses a three-tier web architecture with the machine learning models in a separate Python service, so each part uses the language best suited to it.

PHP runs the website; a separate Python service handles the ML



PlantFolio system architecture · 5 components

The browser only ever talks to the PHP application. When a user uploads a photo for identification or diagnosis, PHP saves it, sends it to the ML service through a REST API, and stores the JSON result in MySQL. The website keeps working even if the ML service is offline.

Layer	Technology	Why it was chosen
Front end	HTML5, CSS3, JavaScript, a responsive CSS framework (Bootstrap)	Web standards required by the course; fast, consistent mobile layouts

Layer	Technology	Why it was chosen
Back end	PHP	Taught in the coursework; well supported on standard hosting
Database	MySQL	Relational data with clear relationships; builds on prior coursework
ML service	Python, Flask, TensorFlow/Keras	Industry-standard ML tools; Flask exposes models as a simple REST API
Model training	Google Colab	Free GPU access for training CNNs
Tooling	VS Code, XAMPP, Git and GitHub, Figma	Free, widely used, and suitable for a portfolio

7. Database Design

The MySQL database uses ten normalised tables; every plant, listing, reminder and prediction traces back to a user through foreign keys.

Table	Purpose	Key fields
users	Accounts and public profiles	id, username, email, password_hash, bio, location, avatar, created_at
collections	Named groups of plants per user	id, user_id, name, is_public
plant_species	Reference data and care information	id, common_name, scientific_name, light, water, humidity, notes
plants	Each individual plant profile	id, user_id, collection_id, species_id, nickname, photo, acquired_on, health_status
health_logs	Dated health timeline per plant	id, plant_id, status, note, photo, source (manual or ML), logged_at

Table	Purpose	Key fields
care_reminders	Recurring care tasks	id, plant_id, task (water, fertilise), interval_days, last_done, next_due
exchange_listings	Plants offered for exchange	id, user_id, plant_id, title, description, location, status (open, closed)
listing_interests	Interest expressed in a listing	id, listing_id, user_id, message, created_at
user_ratings	Trust ratings between traders	id, rater_id, rated_id, listing_id, score (1-5), comment
ml_predictions	Record of every recognition or diagnosis	id, user_id, plant_id, type, predicted_label, confidence, image_path, created_at

Care reminders are calculated from `next_due`, so the dashboard can show due and overdue tasks without any background scheduler. Storing every prediction in `ml_predictions` also creates a record for evaluating model performance later.

8. Machine Learning Components

Both ML features use **transfer learning**: starting from a CNN already trained on millions of general images and adapting it to plants, which gives strong results with modest data and free GPU time.

8.1 Plant recognition

- **Approach:** a pre-trained CNN (such as MobileNetV2, chosen for its speed and small size) fine-tuned on a curated set of common houseplant and garden species. The [Pl@ntNet API](#) is kept as a fallback if the self-trained model's coverage is too narrow.
- **Output:** the top three species suggestions with confidence scores; the user confirms the correct one.

8.2 Disease detection

- **Dataset:** the public PlantVillage dataset of labelled leaf images (healthy and diseased) across a range of crop species.

- **Model:** a CNN built with TensorFlow/Keras, first as a small model trained from scratch to learn the fundamentals, then as a fine-tuned pre-trained model for production use.
- **Training:** Google Colab (free GPU), with data augmentation — rotation, flipping, brightness changes — to make the model more robust.
- **Evaluation:** accuracy, precision, recall and a confusion matrix on a held-out test set, plus a small set of real-world phone photos to check how well it generalises.
- **Output:** the predicted condition, a confidence score and practical care guidance. Predictions below a confidence threshold are shown as *"Unsure — consult an expert."*

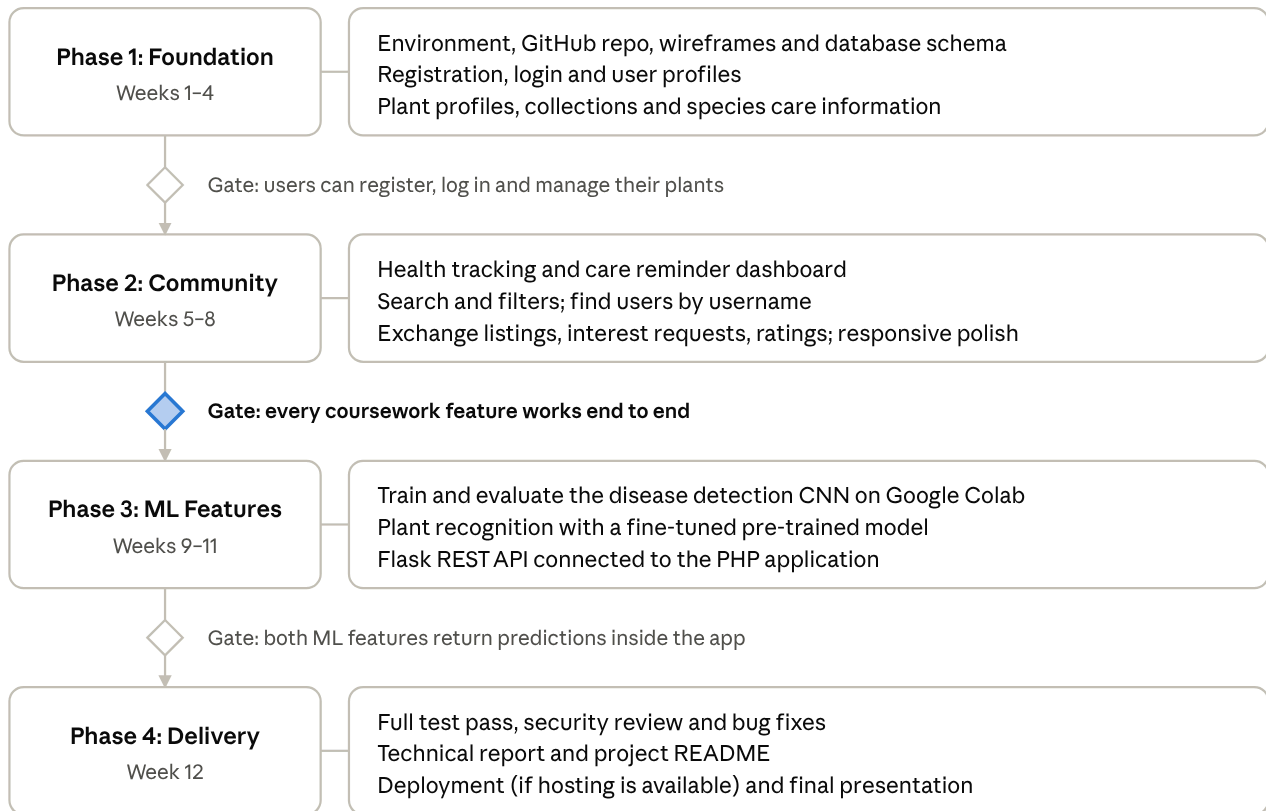
8.3 Scope note

PlantVillage mainly covers food crops such as tomato, potato and apple, photographed on plain backgrounds. Disease detection will therefore list the species it supports, and results on cluttered real-world photos are expected to be lower than on the test set. This limitation will be measured and reported openly rather than hidden.

9. Implementation Plan and Timeline

The project runs in four phases over about 12 weeks, and every coursework requirement is complete by the end of week 8.

Every coursework feature is finished by week 8, before ML work starts



Delivery plan · 4 phases over 12 weeks, 3 gates

Each phase ends at a gate that must be passed before the next begins. If ML work runs late, the core platform is already complete and submittable; week numbers will be aligned to the official submission date once it is confirmed.

10. Development Methodology

The project follows an **iterative, learn-by-doing approach**: each feature is learned and built in the same week, so progress is visible from week one rather than after months of theory.

- **Just-in-time learning.** Each phase begins with the specific skill it needs — HTML/CSS layout before the profile pages, PHP sessions before login, REST APIs before connecting the ML service — using MDN Web Docs, the PHP manual, course material and focused tutorials.
- **Short build cycles.** Every one to two weeks ends with a working, demonstrable increment, reviewed against the feature list before moving on.

- **Version control from day one.** All code lives in a Git repository on GitHub with meaningful commits, giving a clear history of progress and a public portfolio.
- **Build on existing strengths.** Intermediate MySQL and Java knowledge carries over to database design and object-oriented PHP; self-taught Python and current machine learning study feed directly into the ML service.
- **Test as you go.** Each feature is checked manually on desktop and mobile when it is built, with a full test pass in the final phase.

Development tools: VS Code, XAMPP (local Apache, PHP and MySQL), Git and GitHub, Google Colab, and Figma for wireframes.

11. Risk Assessment and Mitigation

The biggest risk is time spent learning new technologies; the phased plan contains it by finishing the required features before any ML work begins.

Risk	Likelihood	Impact	Mitigation
Learning curve for full-stack development slows progress	High	High	Just-in-time learning per phase; start with the simplest version of each feature; weekly progress check
ML features take longer than planned	Medium	Medium	ML is Phase 3, after the core is complete; fall back to a hosted API (Pl@ntNet) for recognition if needed
Disease model performs poorly on real-world photos	High	Medium	Data augmentation, confidence threshold, clear "unsure" message, honest evaluation in the report
Connecting PHP to the Python service is difficult	Medium	Medium	Build a minimal "hello world" API call early in Phase 3 before adding models
Security weaknesses (SQL injection, XSS, unsafe uploads)	Medium	High	Prepared statements, output escaping, CSRF tokens, file type and size checks, password hashing

Risk	Likelihood	Impact	Mitigation
Scope creep from extra ideas	Medium	Medium	Fixed MVP list; new ideas go to a "future work" list
Loss of work	Low	High	Frequent commits to GitHub; database schema kept in version control

12. Evaluation and Success Criteria

The project succeeds when every core feature works end to end and the ML features meet clear, measurable targets.

Area	Success criterion	How it is measured
Functionality	All nine core features in section 4.1 work end to end	Feature checklist and live demonstration
Security	Passwords hashed; all queries parameterised; uploads validated	Code review against a security checklist
Responsiveness	All pages usable at phone, tablet and desktop widths	Browser device-mode testing and real-phone testing
Usability	Test users complete the three key journeys (section 5) unaided	Informal test with 5-10 fellow students
Disease detection	At least 90% accuracy on the held-out PlantVillage test set	Test-set accuracy, precision, recall, confusion matrix
Plant recognition	Correct species appears in the top three suggestions for most test photos	Top-3 accuracy on a small labelled photo set
ML speed	A prediction is returned within a few seconds on the local setup	Timed tests
Code quality	Clear folder structure, reusable components, commented code, regular commits	GitHub history and code review

13. Expected Outcomes and Value

PlantFolio will deliver a working community platform, a trained ML model, and a documented body of work that serves the course and beyond it.

Deliverables

- A fully functional, mobile-responsive PlantFolio website running locally, and online if hosting is available
- A Python ML service with a trained disease detection model and a plant recognition feature
- Source code, database schema and model training notebooks in a public GitHub repository
- A technical report covering design, implementation, ML evaluation and lessons learned
- A live demonstration and presentation

Value

- **For users:** one place to care for, show off and trade plants, with instant help when something goes wrong.
- **For the course:** complete coverage of the brief, demonstrated through a real full-stack application rather than isolated exercises.
- **For professional growth:** hands-on experience across front end, back end, databases, REST APIs and applied machine learning — the combination expected of modern software and AI/ML engineers — resulting in a portfolio project suitable for internships, IEEE student showcases and future study.

14. Conclusion and Request for Approval

PlantFolio meets every requirement of the Plant Care & Exchange Community brief and goes further, combining full-stack web development with applied machine learning in a product people would genuinely use. The phased plan protects the required features first, keeps the ML work contained and measurable, and turns a demanding learning curve into steady, visible progress.

Approval is requested for:

1. The project scope described in sections 4 and 8, including both ML enhancements.
2. A Python (Flask) ML service running alongside the PHP application and connected through a REST API.

3. The four-phase, 12-week delivery plan, with the ML features treated as enhancements beyond the core requirements.

Guidance welcomed on:

- ☐ Whether the final submission should be hosted online or demonstrated locally
- ☐ Any preferred PHP framework, or plain PHP as taught in the course
- ☐ Exact submission and presentation dates, so the timeline can be aligned